



四川大學
SICHUAN UNIVERSITY

Principle of Compiler

luoyining@scu.edu.cn

5.3 Applications of Syntax-Directed Translation

- 自然语言翻译
- 程序设计语言翻译：在5.3节，应用于抽象语法树的构造；在第6章，应用于类型检查和中间代码生成。
- 其他应用场景

Applications of SDD - I

- **Example 2.10** infix to postfix translation

$$\text{exp} \rightarrow \text{exp} + \text{term} \mid \text{exp} - \text{term} \mid \text{term}$$

$$\text{term} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

PRODUCTION	SEMANTIC RULES
$\text{expr} \rightarrow \text{expr}_1 + \text{term}$	$\text{expr.t} = \text{expr}_1.t \parallel \text{term.t} \parallel '+'$
$\text{expr} \rightarrow \text{expr}_1 - \text{term}$	$\text{expr.t} = \text{expr}_1.t \parallel \text{term.t} \parallel '-'$
$\text{expr} \rightarrow \text{term}$	$\text{expr.t} = \text{term.t}$
$\text{term} \rightarrow 0$	$\text{term.t} = '0'$
$\text{term} \rightarrow 1$	$\text{term.t} = '1'$
...	...
$\text{term} \rightarrow 9$	$\text{term.t} = '9'$

PRODUCTION	SEMANTIC RULES
1) $E \rightarrow E_1 + T$	
2) $E \rightarrow E_1 - T$	
3) $E \rightarrow T$	
4) $T \rightarrow (E)$	
5) $T \rightarrow \mathbf{id}$	
6) $T \rightarrow \mathbf{num}$	

Figure 2.10: Syntax-directed definition for infix to postfix translation

Applications of SDD - II

- Consider the following grammar :

$$S \rightarrow (L) \mid a$$
$$L \rightarrow L,S \mid S \qquad ((a) , (a, a))$$

Production	Semantic Rules (打印出括号的对数)
$S' \rightarrow S$	$\text{print}(S.\text{num})$
$S \rightarrow (L)$	$S.\text{num} = L.\text{num} + 1$
$S \rightarrow a$	$S.\text{num} = 0$
$L \rightarrow L_1,S$	$L.\text{num} = L_1.\text{num} + S.\text{num}$
$L \rightarrow S$	$L.\text{num} = S.\text{num}$

Applications of SDD - II

- Consider the following grammar :

$$S \rightarrow (L) \mid \mathbf{a}$$

$$L \rightarrow L,S \mid S \qquad ((a) , (a, a))$$

Production

Semantic Rules (打印括号的最大深度)

$$S' \rightarrow S$$

$$S \rightarrow (L)$$

$$S \rightarrow a$$

$$L \rightarrow L_1,S$$

$$L \rightarrow S$$

Applications of SDD – III

- ! Exercise 5.2.4: This grammar generates binary numbers with a "decimal" point:

$$S \rightarrow L.L \mid L$$
$$L \rightarrow LB \mid B$$
$$B \rightarrow 0 \mid 1$$

Design an L-attributed SDD to compute $S.val$, the decimal-number value of an input string.

For example, the translation of string 101.101 should be the decimal number 5.625.

Hint: use an inherited attribute $L.side$ that tells which side of the decimal point a bit is on.

- 为文法G(S)设计一个S属性的SDD, 计算 $S.val$, 即输入串的十进制数值。

$$S \rightarrow L$$
$$L \rightarrow LB \mid B$$
$$B \rightarrow 0 \mid 1$$

- !! Exercise 5.2.5: Design an S-attributed SDD for the grammar and translation described in Exercise 5.2.4.
- For the SDD in Exercise 5.2.5, give annotated parse tree and dependency graph for the string 101.101.

5.3.1 Construction of Syntax Trees

- 术语表达
 - 语法树 parse tree
 - 抽象语法树 syntax tree, (abstract syntax tree, [AST](#))
- 为表达式构造抽象语法树
 - Example 5.11: S-SDD, 适合在 bottom-up 的语法分析过程中使用;
 - Example 5.12: L-SDD, 适合在 top-down 的语法分析过程中使用。

Example 5.11

左递归文法	计算表达式的值	构造抽象语法树
	$E.val, T.val$: 综合属性 (S-SDD)	$E.node, T.node$: 综合属性 (S-SDD)
PRODUCTION	SEMANTIC RULES	SEMANTIC RULES
1) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$	
2) $E \rightarrow E_1 - T$	$E.val = E_1.val - T.val$	
3) $E \rightarrow T$	$E.val = T.val$	
4) $T \rightarrow (E)$	$T.val = E.val$	
5) $T \rightarrow \mathbf{id}$	$T.val = \mathbf{id}.val$	
6) $T \rightarrow \mathbf{num}$	$T.val = \mathbf{num}.val$	

Example 5.11

- 构造表达式 $a-4+c$ 的抽象语法树。
- S-SDD：在归约的时候执行语义规则。

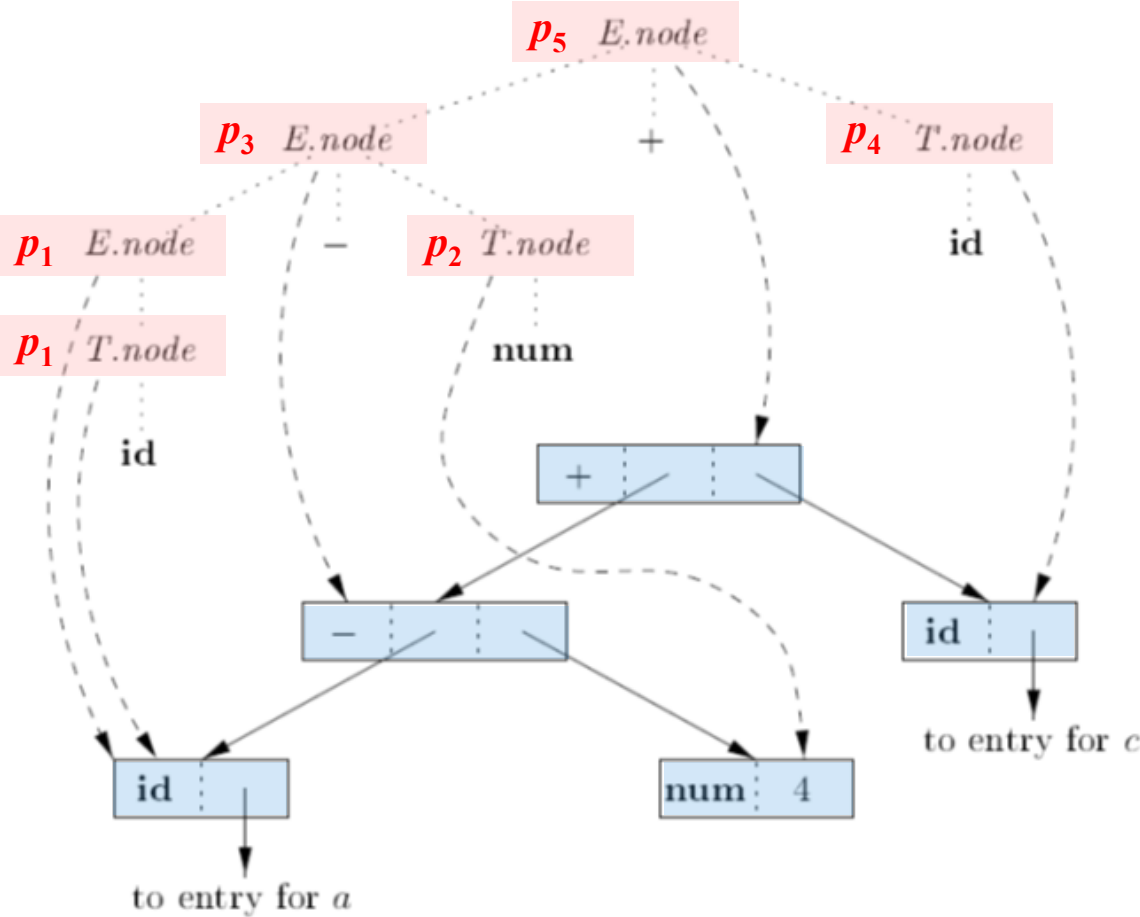


Figure 5.11: Syntax tree for $a - 4 + c$

PRODUCTION	SEMANTIC RULES
1) $E \rightarrow E_1 + T$	$E.node = \mathbf{new} \text{ Node}('+', E_1.node, T.node)$
2) $E \rightarrow E_1 - T$	$E.node = \mathbf{new} \text{ Node}('-', E_1.node, T.node)$
3) $E \rightarrow T$	$E.node = T.node$
4) $T \rightarrow (E)$	$T.node = E.node$
5) $T \rightarrow \mathbf{id}$	$T.node = \mathbf{new} \text{ Leaf}(\mathbf{id}, \mathbf{id.entry})$
6) $T \rightarrow \mathbf{num}$	$T.node = \mathbf{new} \text{ Leaf}(\mathbf{num}, \mathbf{num.val})$

	Symbol	Input	Parsing	Semantic Action
1	\$	a-4+c\$	shift	
2	\$a	-4+c\$	$T \rightarrow \mathbf{id}$	$T.node = \mathbf{new} \text{ Leaf}(\mathbf{id}, \text{entry-}a); \quad // T.node = p_1$
3	\$T	-4+c\$	$E \rightarrow T$	$E.node = T.node; \quad // E.node = p_1$
4	\$E	-4+c\$	shift	
5	\$E-	4+c\$	shift	
6	\$E-4	+c\$	$T \rightarrow \mathbf{num}$	$T.node = \mathbf{new} \text{ Leaf}(\mathbf{num}, 4); \quad // T.node = p_2$
7	\$E-T	+c\$	$E \rightarrow E-T$	$E.node = \mathbf{new} \text{ Node}('-', p_1, p_2); \quad // E.node = p_3$
8	\$E	+c\$	shift	
9	\$E+	c\$	shift	
10	\$E+c	\$	$T \rightarrow \mathbf{id}$	$T.node = \mathbf{new} \text{ Leaf}(\mathbf{id}, \text{entry-}c); \quad // T.node = p_4$
11	\$E+T	\$	$E \rightarrow E+T$	$E.node = \mathbf{new} \text{ Node}('+', p_3, p_4); \quad // E.node = p_5$
12	\$E	\$	accept	

Example 5.12

去除左递归文法	计算表达式的值	构造抽象语法树
	E: <i>val</i> E': <i>inh, syn</i> T: <i>val</i>	E: <i>node</i> E': <i>inh, syn</i> T: <i>node</i>
Production	Semantic Rules	Semantic Rules
1) $E \rightarrow TE'$	$E.val = E'.syn$ $E'.inh = T.val$	$E.node = E'.syn$ $E'.inh = T.node$
2) $E' \rightarrow +TE_1'$	$E_1'.inh = E'.inh + T.val$ $E'.syn = E_1'.syn$	$E_1'.inh = \text{new node}('+', E'.inh, T.node)$ $E'.syn = E_1'.syn$
3) $E' \rightarrow -TE_1'$	$E_1'.inh = E'.inh - T.val$ $E'.syn = E_1'.syn$	$E_1'.inh = \text{new node}('-', E'.inh, T.node)$ $E'.syn = E_1'.syn$
4) $E' \rightarrow \varepsilon$	$E'.syn = E'.inh$	$E'.syn = E'.inh$
5) $T \rightarrow (E)$	$T.val = E.val$	$T.node = E.node$
6) $T \rightarrow \mathbf{id}$	$T.val = \mathbf{id}.val$	$T.node = \text{new Leaf}(\mathbf{id}, \mathbf{id}.entry)$
7) $T \rightarrow \mathbf{num}$	$T.val = \mathbf{num}.val$	$T.node = \text{new Leaf}(\mathbf{num}, \mathbf{num}.val)$

Example 5.12

- 使用不同SDD时 $a-4+c$ 的依赖图：计算表达式的值 vs. 构造抽象语法树
- $E'.inh$ 的语义信息：左运算分量的值 vs. 左运算分量的AST

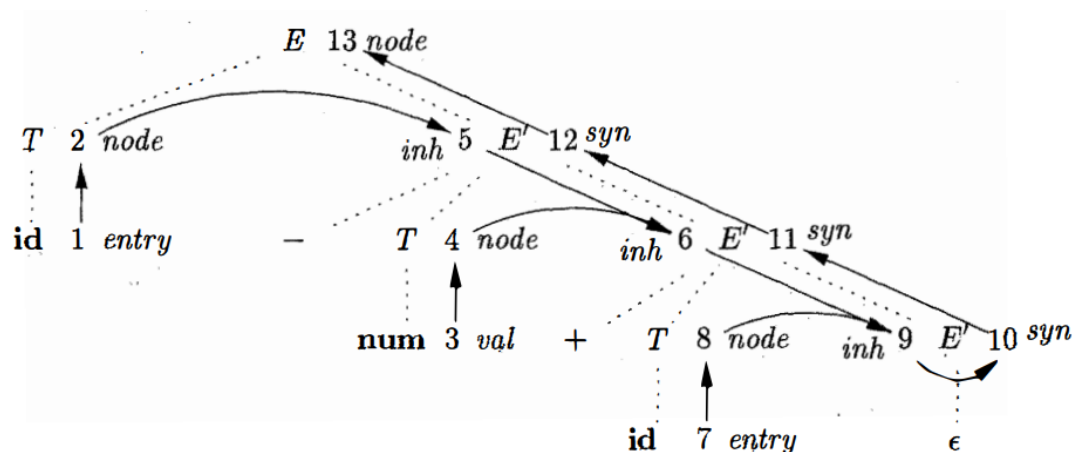
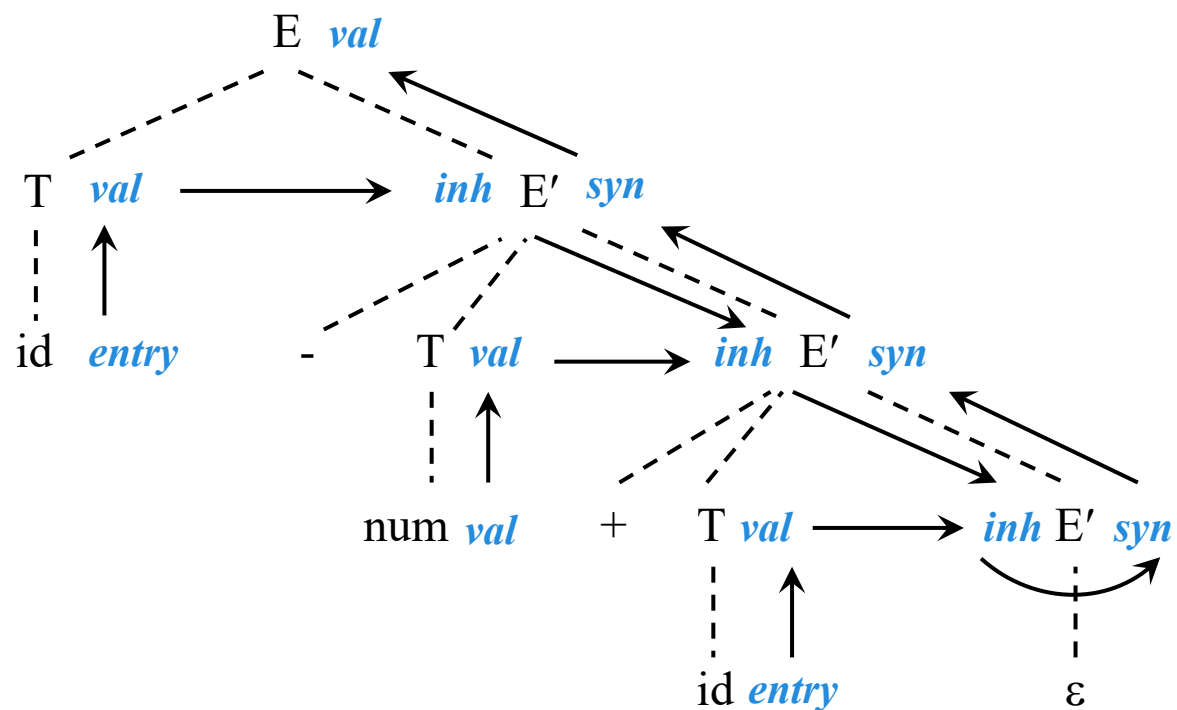


Figure 5.14: Dependency graph for $a-4+c$, with the SDD of Fig. 5.13